

Yes. For a **Doubly Linked List lab**, you can make a set of deletion problems where students are given **node addresses and data**, and they have to **draw the list before and after deletion and update** `prev / next` links.

Doubly Linked List — Deletion Problems

Assume each node has:



Remember:

- First node: `prev = NULL`
- Last node: `next = NULL`
- When deleting a node, both neighboring links may need to be changed.

Problem 1 — Delete Node from Beginning

Given the following doubly linked list:

Address	Data
1000	10
1050	20
1100	30
1150	40
1200	50

Logical order:

1000 → 1050 → 1100 → 1150 → 1200

`HEAD = 1000`

Task

Delete the **first node**.

Students should:

1. Draw the list before deletion.
2. Identify the node to be deleted.
3. Draw the list after deletion.
4. Complete the `prev` and `next` table.

Solution

Before:

```

HEAD
↓
NULL ← [10] ⇌ [20] ⇌ [30] ⇌ [40] ⇌ [50] → NULL
      1000   1050   1100   1150   1200
  
```

Delete node `1000` .

After:

```

HEAD
↓
NULL ← [20] ⇌ [30] ⇌ [40] ⇌ [50] → NULL
      1050   1100   1150   1200
  
```

Address	prev	Data	next
1050	NULL	20	1100
1100	1050	30	1150
1150	1100	40	1200
1200	1150	50	NULL

Important operation

```

HEAD = HEAD->next;
HEAD->prev = NULL;
  
```

Problem 2 — Delete Node from End

Given:

Address	Data
2000	11
2050	22
2100	33
2150	44
2200	55

Logical order:

2000 → 2050 → 2100 → 2150 → 2200

HEAD = 2000

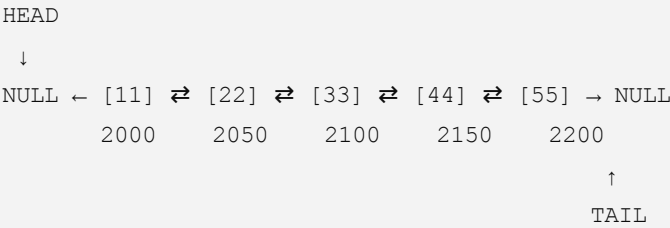
TAIL = 2200

Task

Delete the **last node**.

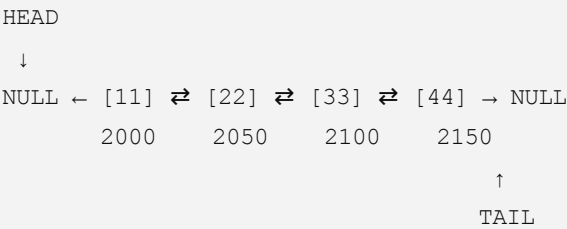
Solution

Before:



Delete 2200 .

After:



Address	prev	Data	next
2000	NULL	11	2050
2050	2000	22	2100
2100	2050	33	2150
2150	2100	44	NULL

Important operation

```
TAIL = TAIL->prev;
TAIL->next = NULL;
```

Problem 3 — Delete a Node After a Given Node

Given:

Address	Data
3000	10
3050	20
3100	30
3150	40
3200	50

Logical order:

3000 → 3050 → 3100 → 3150 → 3200

Task

Delete the node **after node 3050**.

Students must determine:

Node after 3050 = ?

Solution

Node after 3050 is 3100 , containing 30 .

Before:

NULL ← [10] ⇌ [20] ⇌ [30] ⇌ [40] ⇌ [50] → NULL
3000 3050 3100 3150 3200

Delete 3100 .

After:

NULL ← [10] ⇌ [20] ⇌ [40] ⇌ [50] → NULL
3000 3050 3150 3200

Address	prev	Data	next
3000	NULL	10	3050
3050	3000	20	3150
3150	3050	40	3200
3200	3150	50	NULL

Important link changes

3050->next = 3150
3150->prev = 3050

Problem 4 — Delete a Node Before a Given Node

Given:

Address	Data
4000	15
4050	25
4100	35
4150	45

Address	Data
4200	55

Logical order:

4000 → 4050 → 4100 → 4150 → 4200

Task

Delete the node **before node 4150**.

Students must determine:

Node before 4150 = ?

Solution

Node before 4150 is 4100 , containing 35 .

Before:

NULL ← [15] ⇌ [25] ⇌ [35] ⇌ [45] ⇌ [55] → NULL
 4000 4050 4100 4150 4200

After deleting 4100 :

NULL ← [15] ⇌ [25] ⇌ [45] ⇌ [55] → NULL
 4000 4050 4150 4200

Address	prev	Data	next
4000	NULL	15	4050
4050	4000	25	4150
4150	4050	45	4200
4200	4150	55	NULL

Important link changes

4050->next = 4150
 4150->prev = 4050

Problem 5 — Delete After a Given Node: Address Order is Random

This is a slightly harder problem.

Address	Data
5000	10
6200	20
4500	30
7100	40
5800	50

Logical order:

5000 → 4500 → 7100 → 6200 → 5800

Task

Delete the node **after address 4500**.

Do not assume that the numerically next address is the logically next node.

Solution

The node after 4500 is 7100 .

Before:

```
NULL ← [10] ⇄ [30] ⇄ [40] ⇄ [20] ⇄ [50] → NULL
      5000   4500   7100   6200   5800
```

Delete 7100 .

After:

```
NULL ← [10] ⇄ [30] ⇄ [20] ⇄ [50] → NULL
      5000   4500   6200   5800
```

Address	prev	Data	next
5000	NULL	10	4500

Address	prev	Data	next
4500	5000	30	6200
6200	4500	20	5800
5800	6200	50	NULL

Problem 6 — Delete Before a Given Node: Random Addresses

Given:

Address	Data
8000	100
7300	200
9100	300
6500	400
8400	500

Logical order:

8000 → 6500 → 7300 → 8400 → 9100

Task

Delete the node **before address 8400**.

Solution

The node before 8400 is 7300 .

Delete node 7300 .

After:

NULL ← [100] ⇄ [400] ⇄ [500] ⇄ [300] → NULL
 8000 6500 8400 9100

Address	prev	Data	next
8000	NULL	100	6500
6500	8000	400	8400
8400	6500	500	9100
9100	8400	300	NULL

Changed links:

```
6500->next = 8400
8400->prev = 6500
```

Problem 7 — Delete the Second Node

Given:

Address	Data
10000	5
10100	10
10200	15
10300	20
10400	25

Logical order:

10000 → 10100 → 10200 → 10300 → 10400

Task

Delete the **second node**.

Solution

Delete node 10100 .

Before:

NULL ← [5] ⇌ [10] ⇌ [15] ⇌ [20] ⇌ [25] → NULL
10000 10100 10200 10300 10400

After:

NULL ← [5] ⇌ [15] ⇌ [20] ⇌ [25] → NULL
10000 10200 10300 10400

Address	prev	Data	next
10000	NULL	5	10200
10200	10000	15	10300
10300	10200	20	10400
10400	10300	25	NULL

Problem 8 — Delete the Second-Last Node

Given:

Address	Data
11000	10
11100	20
11200	30
11300	40
11400	50

Logical order:

11000 → 11100 → 11200 → 11300 → 11400

Task

Delete the **second-last node**.

Solution

Second-last node = 11300 , data 40 .

After deletion:

NULL ← [10] ⇌ [20] ⇌ [30] ⇌ [50] → NULL
11000 11100 11200 11400

Address	prev	Data	next
11000	NULL	10	11100
11100	11000	20	11200
11200	11100	30	11400
11400	11200	50	NULL

Problem 9 — Delete After a Node and Show All Links

Given:

Address	Data
12000	12
12100	24
12200	36
12300	48
12400	60
12500	72

Logical order:

12000 → 12100 → 12200 → 12300 → 12400 → 12500

Task

Delete the node **after node 12200**.

Students should show:

1. Node being deleted.
2. Node before deleted node.

- 3. Node after deleted node.
- 4. Updated `prev` and `next` .

Solution

Node after `12200` = `12300` .

Node before deleted node = `12200` .

Node after deleted node = `12400` .

After deletion:

```
NULL ← [12] ⇌ [24] ⇌ [36] ⇌ [60] ⇌ [72] → NULL
      12000   12100   12200   12400   12500
```

Updated links:

```
12200->next = 12400
12400->prev = 12200
```

Problem 10 — Challenge: Delete Before a Node

Given:

Address	Data
13000	11
13100	22
13200	33
13300	44
13400	55
13500	66

Logical order:

13000 → 13100 → 13200 → 13300 → 13400 → 13500

Task

Delete the node **before node 13400**.

Solution

Node before 13400 = 13300 .

Delete 13300 .

Before:

```
NULL ← [11] ⇌ [22] ⇌ [33] ⇌ [44] ⇌ [55] ⇌ [66] → NULL
      13000   13100   13200   13300   13400   13500
```

After:

```
NULL ← [11] ⇌ [22] ⇌ [33] ⇌ [55] ⇌ [66] → NULL
      13000   13100   13200   13400   13500
```

Address	prev	Data	next
13000	NULL	11	13100
13100	13000	22	13200
13200	13100	33	13400
13400	13200	55	13500
13500	13400	66	NULL

Summary of Deletion Operations

Operation	Node deleted	Main link changes
Delete from beginning	HEAD node	HEAD = HEAD->next , HEAD->prev = NULL
Delete from end	TAIL node	TAIL = TAIL->prev , TAIL->next = NULL
Delete after a node	P->next	P->next = N->next , then update new node's prev
Delete before a node	P->prev	P->prev = N->prev , then update new node's next

Operation	Node deleted	Main link changes
Delete specific node	Target node	Connect its previous and next nodes

Good classroom exercise format

For each problem, **give students only this information**:

Given: Node addresses, data values, `HEAD`, and logical order.

Task: Delete the specified node and draw the doubly linked list **before and after deletion**. Show the `prev` and `next` values of every remaining node.

This forces students to understand the **actual pointer manipulation**, rather than simply memorizing C code.

Files, images, and data analysis are unavailable until usage resets at 4:17 PM. Continue chatting with text only, or upgrade for more access.

[Claim offer](#)